Method 1: Uploading files from your local file system

Run this cell to select and upload files directly from your computer.

```
from google.colab import files

# This will prompt you to select a file from your computer
uploaded = files.upload()

# Once uploaded, you can load it into Pandas (assuming it's a CSV)
# import pandas as pd
# import io
# filename = list(uploaded.keys())[0]
# df = pd.read_csv(io.BytesIO(uploaded[filename]))
# display(df.head())
```

```
from google.colab import drive
drive.mount('/content/drive')

# Once mounted, you can access your files using their path:
# df = pd.read_csv('/content/drive/MyDrive/path_to_your_file.csv')
```

```
Mounted at /content/drive
```

Method 2: Accessing files from Google Drive

If your dataset is large or you want to keep it saved permanently, upload it to Google Drive first, then run this cell to mount your Drive.

```
import os
for folder, subfolders, files in os.walk('/content/drive/MyDrive'):
  for file in files:
    if file.endswith('.csv'):
      print(os.path.join(folder,file))
```

```
/content/drive/MyDrive/student_outcome_demo_dataset.csv
```

```
import pandas as pd
df=pd.read_csv('/content/drive/MyDrive/student_outcome_demo_dataset
print("Dataset loaded successfully!")
print("Dataset shape:",df.shape)
df.head()
```

```
Dataset loaded successfully!
Dataset shape: (300, 7)
```

	Student_ID	Attendance_Percent	Assignment_Marks	Previous_GPA	St
0	S001	80.3	97.4	7.57	
1	S002	61.4	41.9	5.93	
2	S003	86.5	82.7	8.71	
3	S004	89.2	62.4	5.86	
4	S005	48.7	67.0	5.85	

Next steps: Generate code with df | New interactive sheet

```
# 1. Display all column names
print("Column names:")
print(df.columns.tolist())

# 2. Check the number of rows and columns
print("\nDataset shape:")
print(df.shape)

# 3. Check data types and missing values
print("\nDataset information:")
df.info()

# 4. Count missing values in each column
print("\nMissing values:")
print(df.isnull().sum())

# 5. Count Pass and Fail outcomes
print("\nStudent outcomes:")
print(df["Final_Outcome"].value_counts())
```

```
Column names:
['Student_ID', 'Attendance_Percent', 'Assignment_Marks', 'Previous_G

Dataset shape:
(300, 7)
```

```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 7 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   Student_ID           300 non-null    object 
 1   Attendance_Percent   300 non-null    float64
 2   Assignment_Marks     300 non-null    float64
 3   Previous_GPA         300 non-null    float64
 4   Study_Hours_Per_Day  300 non-null    float64
 5   Missed_Assignments   300 non-null    int64  
 6   Final_Outcome        300 non-null    object 
dtypes: float64(4), int64(1), object(2)
memory usage: 16.5+ KB

Missing values:
Student_ID             0
Attendance_Percent     0
Assignment_Marks       0
Previous_GPA           0
Study_Hours_Per_Day    0
Missed_Assignments     0
Final_Outcome          0
dtype: int64

Student outcomes:
Final_Outcome
Pass    201
Fail     99
Name: count, dtype: int64
```

```
# ==========================================
# STUDENT OUTCOME CLASSIFIER - MODEL BUILDING
# ==========================================

# 1. Import required libraries
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)
import pandas as pd
import matplotlib.pyplot as plt
import os

# 2. Check relationships between features and outcomes
```

```
print("MEDIAN VALUES BY STUDENT OUTCOME")
print(
    df.groupby("Final_Outcome")[
        ["Attendance_Percent", "Assignment_Marks", "Previous_GPA"]
    ].median().round(2)
)

# 3. Select input features (X) and target (y)
features = [
    "Attendance_Percent",
    "Assignment_Marks",
    "Previous_GPA",
    "Study_Hours_Per_Day",
    "Missed_Assignments"
]

X = df[features]
y = df["Final_Outcome"]

# 4. Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining records:", len(X_train))
print("Testing records:", len(X_test))

# 5. Create and train the Random Forest model
model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    class_weight="balanced"
)

model.fit(X_train, y_train)
print("\nModel trained successfully!")

# 6. Predict outcomes for training and testing data
train_predictions = model.predict(X_train)
test_predictions = model.predict(X_test)

# 7. Calculate model accuracy
train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)
```

```
print("\nMODEL ACCURACY")
print("Training accuracy:", round(train_accuracy * 100, 2), "%")
print("Testing accuracy:", round(test_accuracy * 100, 2), "%")

# 8. Display classification report
print("\nCLASSIFICATION REPORT")
print(classification_report(
    y_test,
    test_predictions,
    labels=["Fail", "Pass"],
    zero_division=0
))

# 9. Display confusion matrix
cm = confusion_matrix(
    y_test,
    test_predictions,
    labels=["Fail", "Pass"]
)

cm_df = pd.DataFrame(
    cm,
    index=["Actual Fail", "Actual Pass"],
    columns=["Predicted Fail", "Predicted Pass"]
)

print("\nCONFUSION MATRIX")
print(cm_df)

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Fail", "Pass"]
).plot()

plt.title("Student Outcome Confusion Matrix")
plt.tight_layout()
plt.show()

# 10. Calculate and display feature importance
importance = pd.Series(
    model.feature_importances_,
    index=features
).sort_values(ascending=False)

print("\nFEATURE IMPORTANCE")
print(importance.round(4))

importance.plot(kind="bar")
```

```
plt.title("Feature Importance")
plt.ylabel("Importance Score")
plt.tight_layout()
plt.show()

# 11. Create a table of actual and predicted outcomes
test_results = pd.DataFrame({
    "Actual": y_test,
    "Predicted": test_predictions
})

test_results["Correct"] = (
    test_results["Actual"] == test_results["Predicted"]
)

print("\nFIRST 10 TEST PREDICTIONS")
print(test_results.head(10))

# 12. Save project outputs to Google Drive
output_folder = "/content/drive/MyDrive/Student_Outcome_Classifier"
os.makedirs(output_folder, exist_ok=True)

test_results.to_csv(
    os.path.join(output_folder, "testing_predictions.csv"),
    index=False
)

pd.DataFrame({
    "Actual": y_train,
    "Predicted": train_predictions
}).to_csv(
    os.path.join(output_folder, "training_predictions.csv"),
    index=False
)

cm_df.to_csv(
    os.path.join(output_folder, "confusion_matrix.csv")
)

importance.rename("Importance").to_csv(
    os.path.join(output_folder, "feature_importance.csv")
)

print("\nAll output files saved to:")
print(output_folder)
print("\nPROJECT COMPLETED SUCCESSFULLY!")
```

```
MEDIAN VALUES BY STUDENT OUTCOME
                Attendance Percent  Assignment Marks  Previous GPA
```

```
                                                                    
Final_Outcome                                                       
Fail                          73.5              62.8          5.55
Pass                          76.3              72.0          7.20

Training records: 240
Testing records: 60

Model trained successfully!

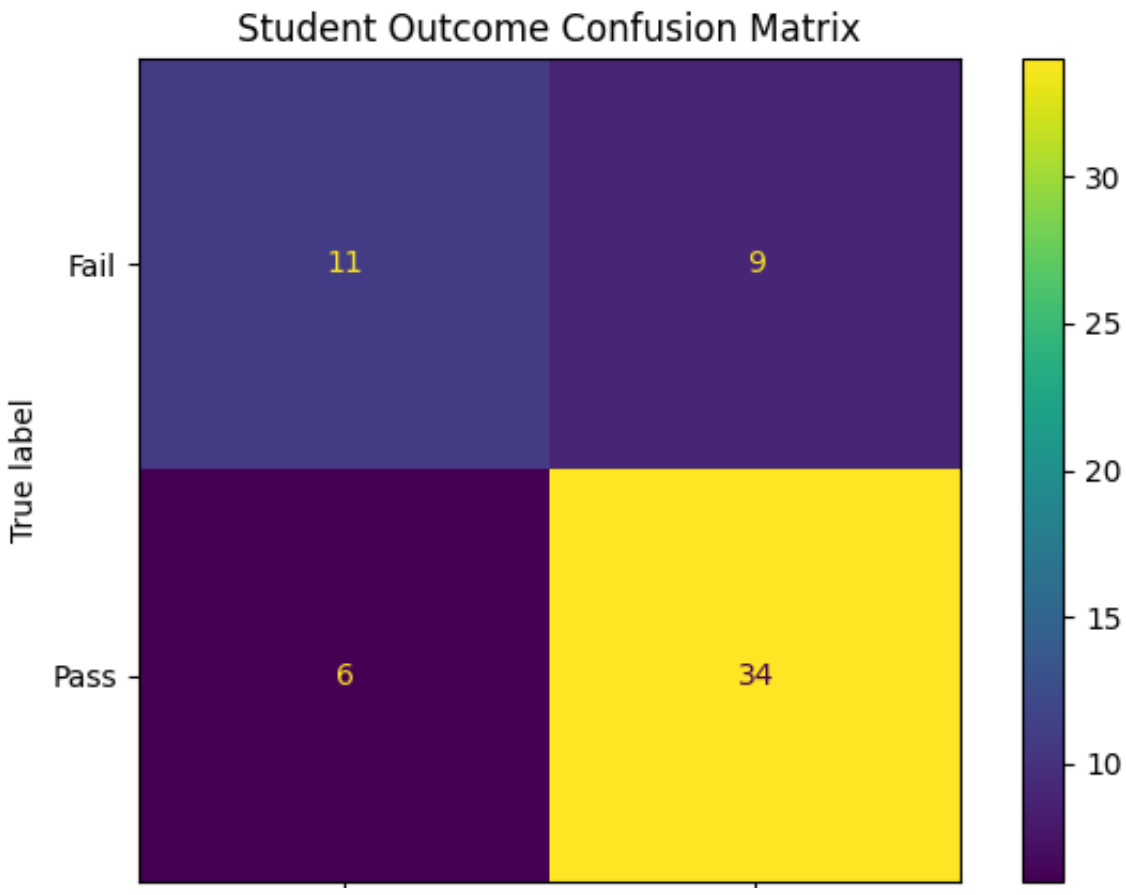
MODEL ACCURACY
Training accuracy: 100.0 %
Testing accuracy: 75.0 %

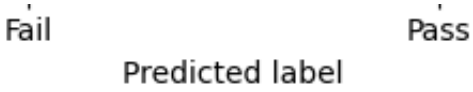
CLASSIFICATION REPORT
              precision    recall  f1-score   support

        Fail       0.65      0.55      0.59        20
        Pass       0.79      0.85      0.82        40

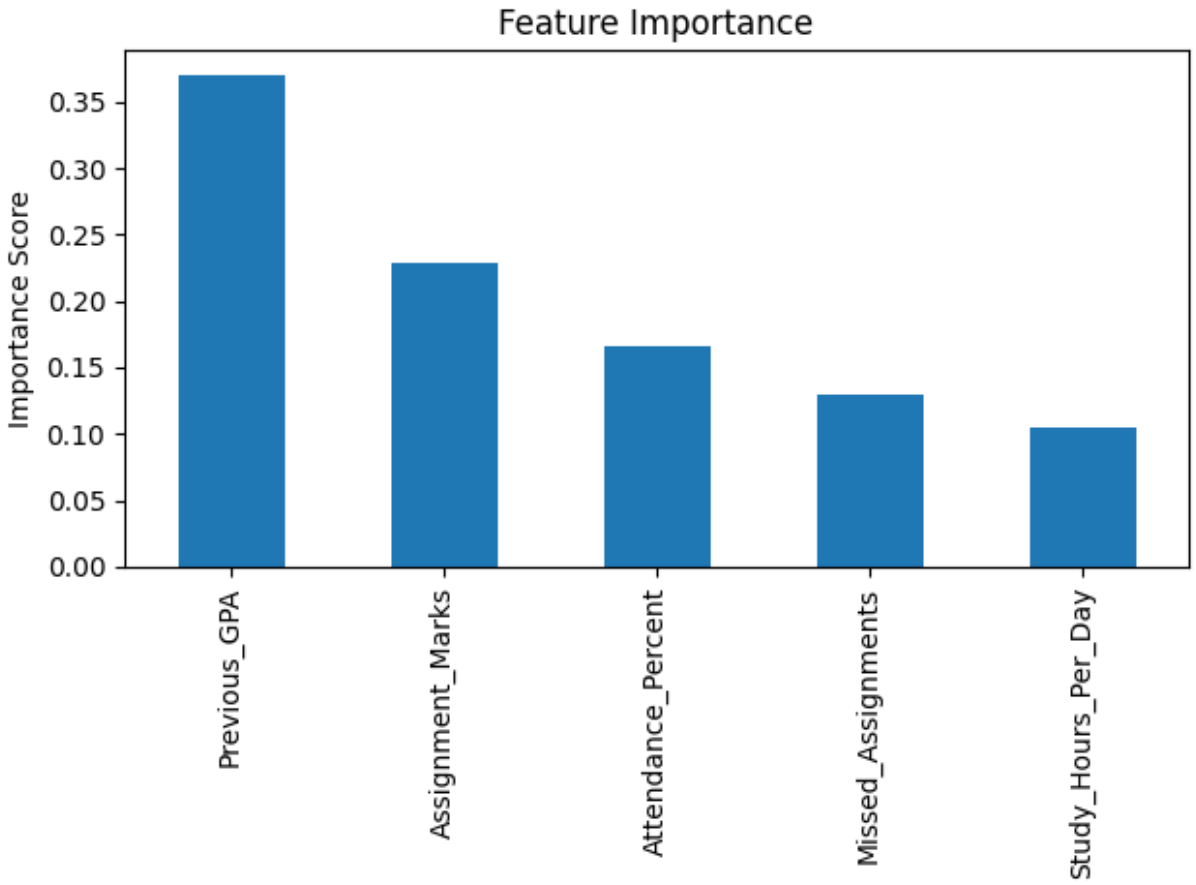
    accuracy                           0.75        60
   macro avg       0.72      0.70      0.71        60
weighted avg       0.74      0.75      0.74        60


CONFUSION MATRIX
             Predicted Fail  Predicted Pass
Actual Fail              11               9
Actual Pass               6              34
```






```
FEATURE IMPORTANCE
Previous_GPA           0.3700
Assignment_Marks       0.2284
Attendance_Percent     0.1664
Missed_Assignments     0.1297
Study_Hours_Per_Day    0.1055
dtype: float64
```




```
FIRST 10 TEST PREDICTIONS
    Actual Predicted  Correct
188   Fail      Fail     True
41    Pass      Pass     True
8     Fail      Fail     True
263   Fail      Pass    False
139   Fail      Pass    False
212   Pass      Pass     True
280   Fail      Fail     True
184   Pass      Pass     True
160   Pass      Pass     True
61    Pass      Pass     True

All output files saved to:
/content/drive/MyDrive/Student_Outcome_Classifier
```

```
PROJECT COMPLETED SUCCESSFULLY!
```